

Raport privind utilizarea instrumentelor de AI în dezvoltarea proiectului NetAuction

Disciplina: Metode de dezvoltare software
Proiect: NetAuction — platformă web de licitații online (ASP.NET Core MVC + React)
Echipa: *Marțole Ilinca-Maria, Ghercă Maria, Mititelu Vlad-George, Simon Albert*

1. Introducere și scop

Acest raport descrie modul în care echipa a folosit instrumente de inteligență artificială pe parcursul dezvoltării aplicației **NetAuction**, o platformă de licitații online realizată în ASP.NET Core MVC (C#), cu un frontend React integrat, notificări în timp real (SignalR), trimitere de email-uri și doi agenți AI integrați în produs.

Scopul raportului nu este să prezinte produsul final, ci **procesul**: ce instrumente AI am folosit, la ce, cu ce câștig de productivitate, ce limitări am întâlnit și cum am verificat rezultatele generate de AI. Distingem clar între:

- **AI ca instrument de dezvoltare** — Copilot, ChatGPT, Claude, Gemini folosite de echipă pentru a scrie/depana cod, a genera documentație și diagrame;
- **AI ca parte din produs** — cei doi agenți Gemini integrați efectiv în aplicație (Evaluator Preț și Ghid Achiziții), care sunt o livrabilă, nu un instrument de lucru.

2. Instrumentele de AI folosite

Instrument	Tip	Unde l-am folosit
GitHub Copilot	Autocomplete în editor	Cod repetitiv: controllere CRUD, view-uri Razor, configurare EF Core, boiler-plate React.
ChatGPT	Asistent conversațional (OpenAI)	Debugging, explicarea erorilor, decizii de arhitectură, întrebări despre ASP.NET Identity și SignalR.
Claude (Cowork)	Asistent conversațional + agent	Generarea diagramelor UML din cod, scrierea documentației, analiza proiectului, verificarea cod ↔ diagrame.
Google Gemini	Model generativ (API)	Instrument de dezvoltare și, mai ales, integrat în produs ca cei doi agenți AI (<code>gemini-2.5-flash</code>).

Fiecare instrument a avut un rol complementar: Copilot pentru viteză la scriere, ChatGPT/Claude pentru raționament și explicații, Gemini pentru funcționalitatea AI din aplicație.

3. Exemple concrete din cod

3.1. Cei doi agenți AI (Gemini) — AiController.cs

Cel mai clar exemplu de cod dezvoltat cu asistență AI sunt cei doi agenți integrați în `Controllers/AiController.cs`. Aici, AI-ul (ChatGPT/Claude) ne-a ajutat la **prompt engineering** — formularea instrucțiunilor trimise modelului — și la structurarea apelului HTTP către API-ul Gemini.

Agent 1 – Evaluator Preț (`POST /api/Ai/sugereaza-pret`): sugerează automat un preț de pornire pornind de la titlul produsului. Rulează în „mod rigid”, cu `temperature: 0.1`, și i se cere modelului să răspundă „DOAR cu numărul întreg”. Răspunsul este apoi curățat cu o expresie regulată (`Regex.Match(..., @"^d+")`) pentru a extrage doar valoarea numerică — o măsură defensivă necesară pentru că modelul nu respectă întotdeauna strict formatul cerut.

Agent 2 – Ghid Achiziții (`POST /api/Ai/consultanta-cumparaturi`): un consultant de cumpărături care rulează în „mod analitic”, cu `temperature: 0.7`. Spre deosebire de primul, acest agent **întâi interoghează baza de date** (licitațiile active din categoria și bugetul cerute) și abia apoi trimite lista reală de produse către Gemini, cu instrucțiuni stricte de formatare HTML (fără Markdown).

Diferența de temperatură între cei doi agenți (0.1 vs 0.7) este o decizie de design clasică ghidată de AI: temperatură mică pentru output determinist (preț), temperatură mare pentru output creativ (recomandări).

3.2. Serviciul de fundal AuctionWorker.cs

Închiderea automată a licitațiilor expirate este realizată de un `BackgroundService` care rulează la fiecare 10 secunde. AI-ul ne-a ajutat să folosim corect pattern-ul ASP.NET Core: `IServiceProvider + CreateScope()` pentru a accesa `DbContext` (scoped) dintr-un serviciu singleton — o capcană frecventă pe care un dezvoltator începător o ratează ușor și pe care ChatGPT a semnalat-o explicit.

3.3. Notificări în timp real — SignalR + Email

În metoda `Licitare` din `LicitatiiController`, atunci când un utilizator este depășit, sistemul trimite simultan o notificare în browser (SignalR, `ReceiveOutbidNotification`) și un email (`EmailSender`). Configurarea hub-ului SignalR și integrarea cu `IHubContext` au fost zone unde sugestiile AI au accelerat semnificativ munca, fiind API-uri pe care nu le cunoșteam în detaliu.

3.4. Frontend React integrat

Componentele React (`Dashboard`, `Details`, `Favorite`) și contextul de favorite (`WatchlistContext.jsx`, bazat pe `localStorage`) au beneficiat de Copilot pentru boilerplate-ul de hooks (`useState`, `useEffect`, `useContext`) și de configurarea Vite pentru build-ul în `wwwroot/dist`.

4. Impact asupra productivității și timpului

Utilizarea AI a avut un impact pozitiv clar asupra ritmului de dezvoltare:

- **Scriere mai rapidă de cod repetitiv.** Controllerele CRUD, view-urile Razor și migrările EF Core au fost scrise mult mai repede cu Copilot, care completa blocuri întregi pornind de la numele metodei și câteva comentarii.
- **Reducerea timpului de documentare.** În loc să căutăm prin documentația oficială ASP.NET Identity, SignalR sau EF Core, am obținut răspunsuri punctuale și exemple direct aplicabile de la ChatGPT/Claude.

- **Onboarding tehnologic accelerat.** Tehnologii noi pentru echipă (SignalR, `BackgroundService`, integrare React într-un proiect MVC) au fost adoptate mai repede.
- **Documentație și diagrame.** Diagramele UML (use case, clase, stări, secvență) și arhitectura au fost generate ca descrieri textuale (Mermaid) direct din cod cu ajutorul Claude, apoi vizualizate și exportate — economisind ore de desenat manual.

Per ansamblu, estimăm că AI-ul a contribuit cel mai mult la **faza de implementare a funcționalităților standard** și la **documentație**, și cel mai puțin la deciziile de arhitectură specifice proiectului, care au rămas în sarcina echipei.

5. Limitări întâlnite și modul de verificare

AI-ul nu a fost infailibil. Am întâlnit limitări reale și am adoptat o atitudine de verificare sistematică:

- **Output care nu respectă formatul cerut.** Chiar și cu `temperature: 0.1` și instrucțiunea „răspunde doar cu numărul”, agentul Evaluator Preț putea returna text suplimentar sau blocuri Markdown. De aceea am adăugat curățare defensivă cu regex în `AiController`.
- **Cod plauzibil dar greșit.** Sugestiile AI erau uneori sintactic corecte dar incorecte în context (de exemplu, accesarea unui `DbContext` scoped dintr-un serviciu singleton, sau validări lipsă). Le-am prins prin testare manuală și erori de runtime.
- **API-uri „inventate” sau învechite.** Ocazional, AI-ul propunea metode care nu existau în versiunea noastră de bibliotecă. Verificarea s-a făcut prin compilare și documentația oficială.
- **Lipsa contextului complet.** AI-ul nu „vedea” întreg proiectul, așa că sugestiile trebuiau adaptate manual la modelele și convențiile noastre (denumiri în limba română: `Licitatie`, `seller_id`, `PretCurent`).

Metode de verificare folosite: compilare și rulare locală, testare manuală a fluxurilor (creare licitație, licitare, închidere automată), revizuire de cod între colegi (code review pe pull request), și verificarea încrucișată cod ↔ diagrame (am descoperit astfel că modelul `Review` exista în cod dar nu era implementat ca funcționalitate, și am corectat diagrama use case în consecință).

6. Proces și workflow

Am integrat AI-ul într-un flux de lucru disciplinat, nu ca înlocuitor al procesului de inginerie software:

- **Issue-uri în GitHub.** Bug-urile și cerințele au fost urmărite ca issue-uri. Codul păstrează urme explicite — de exemplu, comentariile din `LicitatiiController`: „*Issue #5: Sellerul nu poate licita la propriul produs*” și „*Issue #14: Licitățiile expirate dispar din listingul public*”.
- **Pull request-uri și code review.** Modificările au fost integrate prin pull request-uri, cu revizuire între colegi. Codul generat sau asistat de AI a trecut prin același proces de review ca orice alt cod.
- **Dezvoltare incrementală.** Istoricul migrărilor EF Core arată o evoluție incrementală (InitialCreate → AdaugarePozaProfil → AddIsSuspendedUser → AddImageLicitatie → AddCategorie → IncludereLicitatie → Notificari), fiecare pas adăugând o funcționalitate, frecvent cu asistență AI.

- **Testare și evaluarea agenților.** Pentru funcționalitatea AI din produs am planificat teste automate și evaluări (evals) ale agenților — verificând, de exemplu, că Evaluatorul Preț returnează un număr valid pentru o varietate de titluri și că Ghidul Achiziției generează doar HTML valid.

7. Concluzii

Instrumentele de AI au fost un multiplicator de productivitate semnificativ pentru echipa noastră, mai ales la implementarea funcționalităților standard, la depanare și la documentație. În același timp, experiența a confirmat că AI-ul **asistă, dar nu înlocuiește** judecata inginerească: fiecare sugestie a fost verificată prin compilare, testare și code review, iar deciziile de arhitectură și corectitudinea finală au rămas responsabilitatea echipei.

Lecția principală: cea mai bună utilizare a AI-ului nu a fost „scrie-mi aplicația”, ci o colaborare iterativă — prompturi specifice, verificare riguroasă a output-ului și integrarea într-un proces clasic de dezvoltare (issue-uri, pull request-uri, review, testare).